

## Lab 1: Implementing RBAC (Task)

```
1 CREATE ROLE cashier_role;
2
3 CREATE USER mike WITH PASSWORD 'mike123';
4 CREATE USER jon WITH PASSWORD 'jon123';
5
```

Data Output Messages Notifications

CREATE ROLE

Query returned successfully in 50 msec.

```
6 GRANT cashier_role TO mike, jon;
7
8 GRANT SELECT ON film, staff TO cashier_role;
9
10 GRANT SELECT, INSERT ON payment TO cashier_role;
```

Data Output Messages Notifications

GRANT ROLE

Query returned successfully in 48 msec.

```
10
11 REVOKE DELETE ON film, staff from cashier_role;
```

Data Output Messages Notifications

REVOKE

Query returned successfully in 56 msec.

```
2
3 CREATE USER mike WITH PASSWORD 'mike123';
4 CREATE USER jon WITH PASSWORD 'jon123';
5
```

Data Output Messages Notifications

CREATE ROLE

Query returned successfully in 67 msec.

```
8 GRANT SELECT ON film, staff TO cashier_role;
9 GRANT SELECT, INSERT ON payment TO cashier_role;
```

Data Output Messages Notifications

GRANT

Query returned successfully in 51 msec.

## Lab 2: Implementing RLS (Task)

```
1 ALTER TABLE payment ENABLE ROW LEVEL SECURITY;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 53 msec.

```
11 SET ROLE mike;
12 SELECT COUNT(*) FROM payment;
13 SET ROLE jon;
```

Data Output Messages Notifications

|   | count<br>bigint |
|---|-----------------|
| 1 | 7292            |

```
13 SET ROLE jon;
14 SELECT COUNT(*) FROM payment;
```

Data Output Messages Notifications

|   | count<br>bigint |
|---|-----------------|
| 1 | 7304            |

```
3 DROP POLICY IF EXISTS isolate_cashier_payments ON payment;
4 CREATE POLICY isolate_cashier_payments ON payment
5 FOR ALL
6 TO cashier_role
7 USING (
8     staff_id = (SELECT staff_id FROM staff WHERE lower(username) = current_user)
9 );
```

Data Output Messages Notifications

NOTICE: policy "isolate\_cashier\_payments" for relation "payment" does not exist, skipping CREATE POLICY

Query returned successfully in 54 msec.